LockTrace (BCT) Technical Whitepaper — v2.0.0

A High-Throughput Hybrid Proof-of-Work, 3-Phase Atomic CAS Sequencer & Quorum-Attested Layer-1 Blockchain for Censorship-Resistant SocialFi, Decentralized Naming (.bct), and Self-Sovereign Cross-Chain DeFi

Version 2.0.0 — November 2026

LockTrace Core Architecture Team & Decentralized Validator Community

Official Mainnet: https://www.locktrace.space

Genesis Release Authority: `BCT885bc383a03ebed0aa11cedfbfac0d52be3d4080`

Treasury Vault: `BCT_HAZINE_00000000000000000000000000000000000000000`

Bridge Proof-of-Reserve Vault: `BCT_BRIDGE_VAULT_00000000000000000000000000000000`

Base Mainnet wBCT ERC-20: `0xA3C743113a41cFd0146917065d4578337Bb77254`

Abstract

Modern digital communication, identity naming, and financial settlement remain heavily monopolized by centralized platforms that impose unilateral censorship, algorithmic rent-seeking, custodial insolvency risks, and severe latency bottlenecks. Conversely, legacy Layer-1 blockchains suffer from either prohibitive gas fees, sequential execution bottlenecks that freeze user interfaces during block production, or plutocratic validator cartels.

LockTrace (Ticker: BCT) introduces a mathematically provable, high-concurrency Layer-1 blockchain protocol engineered from the ground up to unify **censorship-resistant on-chain social networking (LockTrace Space)**, **sovereign decentralized identity (`.bct` Blockchain Naming System)**, and **non-custodial decentralized finance (DeFi)** within a single zero-trust state machine.

In **Version 2.0.0**, LockTrace introduces the **Hybrid Armored Parallel & 3-Phase Atomic Compare-And-Swap (CAS) Engine** paired with an **$O(1)$ Incremental Delta SQLite Write-Ahead Logging (WAL) Persistence Layer**:

1. **Non-Blocking Sub-5ms Transaction Ingestion:** User HTTP/RPC requests are completely decoupled from Proof-of-Work (PoW) hashing and P2P network broadcasting. Transactions validate ECDSA signatures, replay nonces, and 10^8 integer atomic balances in $< 5\text{ ms}$, queuing into a shared WAL mempool while signaling an autonomous background batch sequencer (`_sequencer_wakeup`).

2. **3-Phase Atomic Compare-And-Swap (CAS) Block Production (Zero-Fork Guarantee):** Candidate blocks are assembled under a microsecond snapshot lock (Phase 1), mined via SHA-256 Proof-of-Work completely outside the ledger lock (Phase 2), and committed via strict Compare-And-Swap tip verification (Phase 3). If an external P2P miner advances the canonical chain during local PoW computation, the CAS Fork Guard deterministically aborts the stale candidate—mathematically eliminating local chain splits and preserving all concurrent mempool arrivals.

3. **$O(1)$ Delta Persistence & Incremental In-Memory Indexing:** Rather than re-serializing historical blocks or rescanning the chain from Genesis, LockTrace v2.0.0 persists only newly sealed delta blocks (`b.index > max_db_idx`) and modified wallet nonces (`dirty_wallets`), while maintaining incremental confirmed-chain balance and social/BNS state overlays.

4. **Dual 70% PoW Proposer / 30% Active Validator Quorum Consensus:** Block rewards and fee pools are split deterministically between the PoW block producer (70%) and cryptographically attested peer validators (30%), enforced with IPv4 `/24` and IPv6 `/48` Sybil subnet deduplication.

5. **Cryptographic Frontend Release Integrity Shield:** Every official web client release (`index.html`, `sw.js`) is hashed via SHA-256, anchored to an IPFS CIDv1 content identifier, signed by the Release Authority (`SECP256k1`), and sealed directly on-chain (`frontend_release`).

Table of Contents

1. [Introduction & Architectural Motivation](#1-introduction--architectural-motivation)

2. [v2.0.0 High-Concurrency Engine: 3-Phase Atomic CAS & O(1) Delta WAL](#2-v200-high-concurrency-engine-3-phase-atomic-cas--o1-delta-wal)

3. [Cryptographic Primitives & 10^8 Atomic Unit Accounting](#3-cryptographic-primitives--108-atomic-unit-accounting)

4. [Dual Hybrid Consensus: 70% PoW Proposer + 30% Validator Quorum](#4-dual-hybrid-consensus-70-pow-proposer--30-validator-quorum)

5. [Hybrid Time-and-Quota Batch Sequencer](#5-hybrid-time-and-quota-batch-sequencer)

6. [LockTrace Space: Native On-Chain SocialFi & Creator Royalties](#6-locktrace-space-native-on-chain-socialfi--creator-royalties)

7. [BCT Naming System (BNS): Sovereign `.bct` Identity & Marketplace](#7-bct-naming-system-bns-sovereign-bct-identity--marketplace)

8. [Tokenomics, Deflationary Mechanics & Supply Invariants](#8-tokenomics-deflationary-mechanics--supply-invariants)

9. [Cross-Chain Interoperability: 1:1 Base Mainnet wBCT Bridge](#9-cross-chain-interoperability-11-base-mainnet-wbct-bridge)

10. [On-Chain Frontend Integrity Shield & Tamper Detection](#10-on-chain-frontend-integrity-shield--tamper-detection)

11. [Multi-Layered Cyber Defense: Anti-DDoS, Replay & Sybil `/24` Hardening](#11-multi-layered-cyber-defense-anti-ddos-replay--sybil-24-hardening)

12. [Decentralized Node Runner (`node_runner.py`) & P2P Topology](#12-decentralized-node-runner-node_runnerpy--p2p-topology)

13. [Comprehensive Threat Model & Formal Verification Matrix](#13-comprehensive-threat-model--formal-verification-matrix)

14. [Roadmap & Protocol Evolution](#14-roadmap--protocol-evolution)

---

1. Introduction & Architectural Motivation

1.1 Structural Failures of Legacy Web2 and Web3 Systems

Contemporary digital networks fail users across four critical dimensions:

- **Unilateral Platform Censorship:** Centralized social networks arbitrarily de-platform creators, shadow-ban discourse, and alter social graphs without cryptographic accountability.

- **Predatory Intermediary Economics:** Creators generate the entirety of user engagement on Web2 social networks yet receive zero direct protocol royalties from likes, comments, or reposts.

- **L1 Execution Stalls & UI Freezing:** Traditional single-threaded blockchain nodes lock ledger state during block assembly and persistence, causing severe request queueing, HTTP timeouts, and UI freezes when thousands of users transact simultaneously.

- **Unverified Client Supply-Chain Attacks:** Most Web3 decentralized applications (DApps) serve unverified JavaScript from centralized CDNs; a single DNS hijack or compromised CDN cache allows attackers to drain non-custodial wallets silently.

1.2 The LockTrace v2.0.0 Solution

LockTrace v2.0.0 resolves these challenges through an end-to-end verifiable stack:

- **Protocol-Level SocialFi & Naming:** Posts, quote-reposts, likes, comments, follows, and `.bct` domain operations are first-class Layer-1 transactions—eliminating smart-contract reentrancy bugs while executing instant on-chain royalty splits.

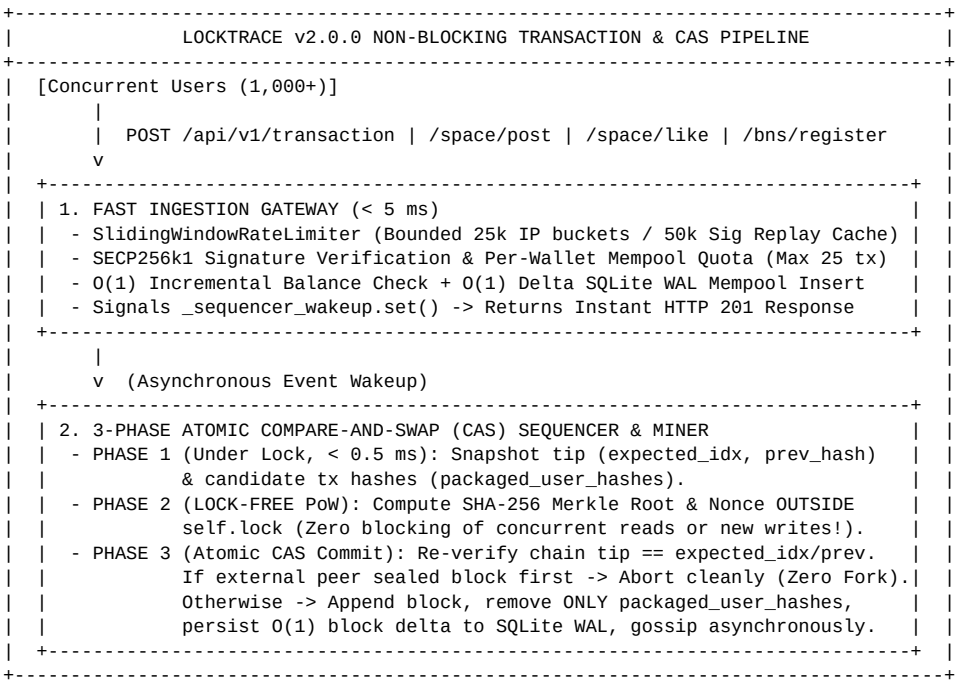
- **Lock-Free Asynchronous PoW & O(1) Delta Storage:** High-frequency user actions never wait on PoW mining or full-chain disk writes, enabling thousands of concurrent users to interact with sub-5ms responsiveness.

- **Zero-Knowledge Browser Custody + On-Chain Code Verification:** Private keys never leave the client's WebCrypto sandbox, and the client verifies its own SHA-256 + ECDSA release signature against the blockchain before unlocking wallet operations.

---

2. v2.0.0 High-Concurrency Engine: 3-Phase Atomic CAS & O(1) Delta WAL

A foundational breakthrough in LockTrace v2.0.0 is the elimination of `O(N)` full-chain serialization and synchronous lock contention under heavy multi-user load.



2.1 Why Legacy Blockchain Nodes Freeze Under Load

In naive implementations, reaching a batch threshold inside a user's HTTP request triggers synchronous Proof-of-Work mining while holding the global ledger mutex, followed by serializing the entire blockchain (`[b.to_dict() for b in chain]`) to disk. At 10,000+ blocks and 1,000+ concurrent users, this creates $O(N)$ disk I/O amplification and multi-second request stalls.

2.2 3-Phase Atomic Compare-And-Swap (CAS) Block Sealing

LockTrace v2.0.0 separates transaction admission from block sealing via a strict 3-phase protocol:

1. **Phase 1 — Canonical Snapshot:** Under `self.lock`, the sequencer captures:

$$\text{Snapshot} = \left(h_{\text{expected}} = |\mathcal{C}|,\; H_{\text{prev}} = \mathcal{C}[-1].\text{hash},\; \mathcal{P} = \{\text{tx.hash}() \mid \text{tx} \in \text{Mempool}\}\right)$$

along with deterministic 70/30 reward transactions.

2. **Phase 2 — Lock-Free Proof-of-Work:** Guarded by `self._mining_lock` (preventing redundant local PoW threads) but **outside** `self.lock`, the worker searches for a nonce satisfying $\text{SHA256}(\text{Header}) < \text{Target}(D)$. During this entire window, HTTP workers process reads and new transactions with zero lock wait.

3. **Phase 3 — Atomic CAS Commit & Fork Guard:** Upon solving the puzzle, the worker re-acquires `self.lock`, synchronizes with SQLite WAL, and evaluates the Compare-And-Swap invariant:

$$\text{CommitAllowed} \iff \left(|\mathcal{C}| == h_{\text{expected}}\right) \land \left(\mathcal{C}[-1].\text{hash} == H_{\text{prev}}\right) \land \left(\mathcal{P} \subseteq \text{Mempool}_{\text{live}}\right)$$

- **Zero Chain Split (Fork Immunity):** If an external PoW miner (`submit_mined_block`) or P2P peer advanced the chain while Phase 2 was hashing, the CAS condition evaluates to `False` and the local candidate is deterministically discarded (`[CAS FORK GUARD]`).

- **Zero Transaction Loss:** Upon commit, the mempool is pruned strictly via set difference:

$$\text{Mempool}_{\text{next}} = \{\text{tx} \in \text{Mempool}_{\text{live}} \mid \text{tx.hash}() \notin \mathcal{P}\}$$

preserving 100% of transactions that arrived while PoW was in progress.

2.3 $O(1)$ Delta SQLite WAL Persistence & Multi-Worker Sync

- **Delta Block Writes:** `SQLiteStorage.get_max_block_index()` queries the highest persisted `block_index` in `O(1)` time. Only blocks with `index > max_db_idx` are serialized and inserted into the `blocks` and `tx_lookup` tables.

- **Dirty Wallet Upserts:** The engine tracks `_saved_wallet_nonces` as `(public_key, nonce)` tuples per address, writing only mutated accounts (`dirty_wallets`) per transaction rather than rewriting the entire wallet registry.

- **$O(\Delta)$ Cross-Worker Synchronization (`load_delta_state`):** When Gunicorn worker processes detect a bumped `state_revision`, they verify their local tip hash against `SELECT hash FROM blocks WHERE block_index = ?` and fetch only `WHERE block_index > ?`, reducing multi-process synchronization overhead from $O(|\mathcal{C}|)$ to $O(\Delta)$.

---

3. Cryptographic Primitives & 10^8 Atomic Unit Accounting

3.1 SECP256k1 Elliptic Curve Digital Signatures

All account identities, transactions, node heartbeats, block submissions, and frontend releases are authenticated via the Koblitz curve `secp256k1`:

$$y^2 \equiv x^3 + 7 \pmod p, \quad p = 2^{256} - 2^{32} - 977$$

- **Deterministic Signing (RFC 6979):** Signatures use HMAC-SHA256 deterministic k-nonce generation, eliminating random number generator (CSPRNG) entropy failures.

- **Address Derivation:** Given an uncompressed 64-byte public key $Q = X \parallel Y$:

$$\text{Address} = \text{"BCT"} \parallel \text{SHA256}\left(\text{SHA256}(Q)_{\text{hex}}\right)[0..39]$$

yielding a 43-character collision-resistant address (e.g., `BCT885bc383a03ebed0aa11cedfbfac0d52be3d4080`).

3.2 BIP-39 12-Word Deterministic Key Derivation

Wallets are generated client-side from 128 bits of WebCrypto entropy mapped to 12 BIP-39 English mnemonic words (2,048-word dictionary with 4-bit SHA-256 checksum), stretched via PBKDF2-HMAC-SHA512 (2,048 iterations).

3.3 10^8 Integer Atomic Unit Ledger (Zero IEEE-754 Drift)

At a maximum supply of $1,000,000,000,000\text{ BCT}$ with 8 decimal places of precision, standard 64-bit IEEE-754 floating-point arithmetic (`float64`, 53-bit significand $\approx 9 \times 10^{15}$) suffers from rounding drift when adding micro-royalties (0.002 BCT) to trillion-scale treasury balances ($10^{12} \times 10^8 = 10^{20}$ atomic units).

LockTrace executes all ledger accounting in arbitrary-precision Python `int` atomic units:

$$1\text{ BCT} = 100,000,000\text{ Atomic Units } (10^8)$$

$$\text{Units}(v) = \text{RoundHalfUp}(\text{Decimal}(v) \times 10^8)$$

Moreover, `_ensure_ledger_index()` caches the confirmed-chain unit balance map (`_confirmed_units_map`) and advances it incrementally per block, applying active mempool transactions as an in-memory overlay in $O(|\text{Mempool}|)$ time.

3.4 Binary Merkle Trees & Light-Client SPV Proofs (Block v2)

Every v2 block header commits to the binary SHA-256 Merkle root of its ordered transaction hashes:

$$\text{MerkleParent}(L, R) = \text{SHA256}(\text{ASCII}(L) \parallel \text{ASCII}(R))$$

Any browser or light client can verify transaction inclusion via `/api/v1/explorer/merkle-proof/<tx_hash>` in $O(\log_2 N)$ steps without downloading full block bodies.

---

4. Dual Hybrid Consensus: 70% PoW Proposer + 30% Validator Quorum

LockTrace combines permissionless Proof-of-Work block production with cryptographic peer validator attestations.

4.1 Block Header v2 Specification & Proof-of-Work

A v2 block header is serialized deterministically as:

$$\text{HeaderPayload} = \text{JSON}\left(\{\text{index}, \text{merkle_root}, \text{miner}, \text{nonce}, \text{previous_hash}, \text{timestamp}, \text{version}: 2\}\right)$$

A block is valid only if:

$$\text{SHA256}(\text{HeaderPayload})\text{ starts with } \underbrace{\text{"000...0"}}_{D\text{ zeroes}}$$

where base difficulty $D = 3$ (scaling adaptively to $D + 1$ when ≥ 8 external peer miners are concurrently active).

4.2 Deterministic 70% / 30% Reward Pool Split

Each block distributes a reward pool consisting of the base treasury subsidy (0.05 BCT) plus the miner share of transaction fees:

$$\text{TotalPool}_u = \text{Subsidy}_u + \left\lfloor\frac{\text{TransferFees}_u}{2}\right\rfloor + \text{SocialAndBnsFees}_u$$

- **Case A (Active Verified Peer Validators Online):**

- **30% Validator Quorum Pool:** $\text{ValPool}_u = \lfloor 0.30 \times \text{TotalPool}_u \rfloor$, divided equally among up to 10 eligible, Sybil-deduplicated peer validators (`validator_reward`).

- **70% Proposer Share:** $\text{Proposer}_u = \text{TotalPool}_u - \text{ActualValTotal}_u$, paid to the miner who solved the PoW puzzle (`mining_reward`).

- **Case B (No External Eligible Validators Online):**

- 100% of TotalPool_u is awarded to the PoW block producer.

4.3 Zero-Trust External Mining (`getblocktemplate` / `submitblock`)

External nodes running `node_runner.py` request candidate templates via `/api/v1/mining/template`, compute SHA-256 PoW locally, sign the solved header (`SUBMIT_BLOCK:{index}:{hash}:{nonce}:{miner}`), and submit via `/api/v1/mining/submit`. The core verifier independently reconstructs the mempool transaction set, re-verifies the exact 70/30 reward transactions, checks the Merkle root, and validates the PoW hash before committing.

---

5. Hybrid Time-and-Quota Batch Sequencer

To prevent empty-block bloat while maintaining rapid finality for both financial transfers and high-frequency social micro-actions, LockTrace employs a dual-trigger **Time + Quota Hybrid Batch Sequencer** (`_check_batch_ready`):

```
| Transaction Class | Quota Trigger (Instant Seal) | Time-Window Trigger (Max Wait) | Fallback Safety Window |
|:------------------|:-----------------------------|:-------------------------------|:-----------------------|
| **Space Likes (`space_like`)** | $\ge 15$ likes in mempool | $20.0\text{ seconds}$ | $15.0\text{ seconds}$ |
| **Space Comments (`space_comment`)** | $\ge 10$ comments in mempool | $50.0\text{ seconds}$ | $15.0\text{ seconds}$ |
| **General Transfers / Posts / BNS** | $\ge 25$ total txs in mempool | $20.0\text{ seconds}$ | $15.0\text{ seconds}$ |
```

When any quota or time window is reached, `_seal_or_queue_mempool()` triggers `_sequencer_wakeup.set()`, waking the background `LockTrace-BatchSequencer` thread to seal all pending mempool transactions into a single Merkle PoW block.

---

6. LockTrace Space: Native On-Chain SocialFi & Creator Royalties

LockTrace Space embeds a complete decentralized social network directly into Layer-1 transaction primitives.

6.1 Social Transaction Economics & Direct Creator Payouts

```
| Transaction Type | User Cost | Creator / Target Royalty | Treasury Return (Deflationary) | Miner & Validator Fee |
|:-----------------|:----------|:-------------------------|:-------------------------------|:----------------------|
| **Publish Post (`space_post`)** | $0.10\text{ BCT}$ | — (Earns future royalties) | $0.08\text{ BCT}$ (80%) | $0.02\text{ BCT}$ (
| **Quote / Repost (`space_post`)** | $0.10\text{ BCT}$ | Embedded `@handle (BCT...)` attribution card | $0.08\text{ BCT}$ (80%) |
| **Like Post (`space_like`)** | $0.01\text{ BCT}$ | **$+0.002\text{ BCT}$ (20% to Author)** | $0.007\text{ BCT}$ (70%) | $0.001\te
| **Comment (`space_comment`)** | $0.02\text{ BCT}$ | **$+0.004\text{ BCT}$ (20% to Author)** | $0.014\text{ BCT}$ (70%) | $0.002\
| **Follow User (`space_follow`)** | $0.02\text{ BCT}$ | **$+0.004\text{ BCT}$ (20% to Target)** | $0.014\text{ BCT}$ (70%) | $0.00
```

6.2 Native Quote-Repost Attribution & Identity Modes

- **Cryptographic Attribution:** When a user quotes/reposts a thought (■ Al■nt■: `@handle (BCT...)`), the original author's BNS handle and wallet address are permanently embedded in the on-chain payload and rendered as an interactive profile-linked quote card.

- **Sovereign Identity Selection:** Users can publish under their verified `.bct` handle, their raw BCT address, or in cryptographic `anonymous` mode while still receiving on-chain author royalties.

---

7. BCT Naming System (BNS): Sovereign `.bct` Identity & Marketplace

LockTrace includes a native, zero-smart-contract domain registry and secondary peer-to-peer marketplace for `.bct` handles:

7.1 Tiered Registration Pricing (100% Permanent Ownership, Zero Renewal Fees)

- **3 Characters (Ultra-Rare, e.g., `ali.bct`):** 25.00 BCT

- **4 Characters (Rare, e.g., `meta.bct`):** 10.00 BCT

- **5+ Characters (Standard, e.g., `satoshi.bct`):** 2.00 BCT

7.2 Native Marketplace & Bidding Operations

All BNS lifecycle operations are atomic Layer-1 transactions:

- `bns_register`: Mints a `.bct` handle (80% returns to Treasury Vault, 20% to Miner/Validator Fee Pool).

- `bns_set_primary`: Binds a user's primary reverse-resolution handle across Explorer, Wallet, and Space feeds.

- `bns_set_price` & `bns_buy`: Lists a domain for fixed-price instant settlement with a 1% protocol fee.

- `bns_offer`, `bns_accept_offer`, `bns_cancel_offer`: Full on-chain peer-to-peer bid book with automatic superseding of prior active offers from the same bidder.

---

8. Tokenomics, Deflationary Mechanics & Supply Invariants

```
+-------------------------------------------------------------------------------+
|                    LOCKTRACE (BCT) MACROECONOMIC SUPPLY LEDGER                |
+-------------------------------------------------------------------------------+
|  Maximum Fixed Supply          : 1,000,000,000,000.00000000 BCT (1 Trillion)  |
|  Atomic Precision              : 10^8 units per BCT (Zero float64 drift)      |
|  New Wallet Welcome Grant      : 1.00 BCT (Capped at max 25 grants/hour global)|
|  Standard Wire Transfer Fee    : 1.00% (50% Treasury Return / 50% Miner+Quorum)|
|  Base Block Subsidy            : 0.05 BCT (70% PoW Proposer / 30% Quorum Pool)|
|  Provable Burn Address         : BCT0000000000000000000000000000000000000000  |
+-------------------------------------------------------------------------------+
```

8.1 Closed-Loop Circular Treasury Economy

Unlike inflationary networks that mint unbounded new tokens, LockTrace's entire $1,000,000,000,000\text{ BCT}$ supply is created in Block #0 (`GENESIS_VAULT` $\to$ `TREASURY_ADDRESS`).

- **Inflows to Treasury (Deflationary Contraction):** 50% of all 1% wire transfer fees, 80% of Space post & BNS registration fees, and 70% of Space like/comment/follow fees flow back into `TREASURY_ADDRESS`, removing BCT from circulating supply.

- **Outflows from Treasury:** Only verified new wallet grants (1.00 BCT, strictly rate-limited to $\le 25\text{ grants/hour}$ globally) and PoW/Validator block subsidies (0.05 BCT/block) leave the Treasury.

- **Circulating Supply Invariant:**

$$\text{CirculatingSupply} = \text{TotalSupply} - \text{TreasuryBalance} - \text{BridgeVaultLocked} - \text{BurnedAndLostKeys}$$

---

9. Cross-Chain Interoperability: 1:1 Base Mainnet wBCT Bridge

LockTrace connects natively to the Ethereum Layer-2 ecosystem via a 1:1 collateralized bridge on **Base Mainnet (Chain ID 8453)**:

- **Official WrappedBCT (`wBCT`) Contract:** `0xA3C743113a41cFd0146917065d4578337Bb77254` (Verified Solidity 0.8.20).

- **L1 $\to$ Base (Lock & Mint):** Users lock native BCT into `BCT_BRIDGE_VAULT_0000000000000000000000000000000000`. The automated relayer daemon (`bridge_relayer.py`) verifies the L1 finality and invokes `mintFromBridge(to, amount, locktraceTxHash)` on Base Mainnet with idempotent replay protection.

- **Base $\to$ L1 (Burn & Unlock):** Users invoke `burnToBridge(amount, locktraceRecipient)` on Base. The L1 engine verifies the EVM burn receipt (`verify_evm_burn_receipt`), enforces strict deduplication on `evm_tx_hash`, and releases native BCT from `BCT_BRIDGE_VAULT` to the recipient's L1 wallet.

---

10. On-Chain Frontend Integrity Shield & Tamper Detection

To defend against DNS hijacking, rogue reverse proxies, CDN cache poisoning, and Man-in-the-Middle (MitM) script injection:

1. **Release Signing & On-Chain Sealing:** Every release computes canonical SHA-256 digests of `frontend/index.html` and `frontend/sw.js` plus an IPFS CIDv1 content hash, signed by `RELEASE_AUTHORITY_PUBKEY` (`BCT885bc383a03ebed0aa11cedfbfac0d52be3d4080`) over:

$$\text{Payload} = \text{"LOCKTRACE_FRONTEND_RELEASE:"} \parallel \text{Version} \parallel \text{SHA256}_{\text{index}} \parallel \text{SHA256}_{\text{sw}} \parallel \text{CIDv1}$$

and seals the manifest on-chain in a `frontend_release` block transaction.

2. **Stat-Memoized Server Gate (`_inspect_local_frontend_integrity`):** Before serving `/`, the node verifies the local file hashes and ECDSA signature (memoized in $O(1)$ memory via `st_mtime_ns` and `st_size`). Any unauthorized byte modification on disk immediately triggers an `HTTP 403 Security Lock`.

3. **Persistent Client-Side Security Discrepancy Indicator:** The browser independently cross-checks `/api/v1/frontend/integrity` against the Release Authority public key. If any discrepancy occurs (such as stale PWA cache, proxy tampering, or node desynchronization), a persistent **WARNING** badge appears next to the **Mainnet** header indicator. Clicking it opens a detailed security diagnostic modal explaining the exact vulnerability vector (MitM, cache mismatch, or fork) with a one-click **Purge Cache & Resync** recovery action.

---

11. Multi-Layered Cyber Defense: Anti-DDoS, Replay & Sybil `/24` Hardening

LockTrace v2.0.0 incorporates five interlocking security shields against network and application-layer attacks:

1. **Per-Wallet & Global Mempool Quotas (`MAX_PENDING_TX_PER_WALLET = 25`):** A single wallet address can hold at most 25 unconfirmed transactions in the mempool at once (with a global ceiling of `MAX_GLOBAL_MEMPOOL_SIZE = 5000`). Botnets attempting mempool exhaustion are rejected at the signature gateway before touching disk or consensus state.

2. **Bounded Sliding-Window Rate Limiter:** `SlidingWindowRateLimiter` enforces strict per-IP and per-scope request windows while bounding internal memory tables (`MAX_BUCKETS = 25,000`, `MAX_SIG_CACHE = 50,000`) to guarantee zero RAM exhaustion under spoofed-IP DDoS floods.

3. **Full-History + Mempool Signature Replay Immunity:** `_used_signatures_set` indexes every ECDSA signature ever committed from Block #0 through the live mempool in $O(1)$ hash-set lookup time, rendering transaction replay mathematically impossible.

4. **IPv4 `/24` & IPv6 `/48` Sybil Subnet Deduplication (`_extract_node_host_key`):** Active peer miners and validator quorum candidates are deduplicated across both wallet addresses and normalized network subnets (`subnet24:A.B.C.0/24` for IPv4, `subnet48` for IPv6). An attacker provisioning 100 cloud VMs within the same `/24` datacenter subnet receives the consensus weight and validator reward share of at most **1 node**.

5. **SSRF-Hardened P2P Gossip (`_is_safe_peer_endpoint`):** Outbound block gossip resolves DNS records and blocks loopback, link-local, multicast, and private RFC-1918 IP ranges in production, preventing Server-Side Request Forgery (SSRF) via malicious node endpoint registrations.

---

12. Decentralized Node Runner (`node_runner.py`) & P2P Topology

`node_runner.py` is a self-contained, zero-dependency Python 3 validator, PoW miner, and verified web wallet server:

- **Local Sovereign Mode (`python node_runner.py --local --open`):** Verifies frontend SHA-256 integrity, syncs the full archival chain from Genesis, and serves the non-custodial Web Wallet at `http://127.0.0.1:5050`—ensuring users can always access their funds and Space feed even if `locktrace.space` is unreachable.

- **Public Server / Validator Mode (`python node_runner.py --server --port 5050 --wallet BCT...`):** Exposes P2P gossip endpoints, submits cryptographic block attestations (`/api/v1/nodes/attest`) to earn the **30% Validator Quorum Pool**, and mines v2 Merkle PoW blocks to earn the **70% Proposer Reward**.

---

13. Comprehensive Threat Model & Formal Verification Matrix

```
| Attack / Failure Vector | Threat Mechanism | LockTrace v2.0.0 Defense Mechanism | Complexity / Bound |
|:------------------------|:-----------------|:-----------------------------------|:-------------------|
| **Concurrent PoW & P2P Race (Chain Split)** | Local sequencer mines block while external miner submits same height | **3-Phase At
| **High-Concurrency HTTP Freeze** | 1,000+ users trigger batch threshold simultaneously | Non-blocking `_sequencer_wakeup` + lock-
| **Disk I/O Amplification Crash** | Rewriting all historical blocks on every transaction | **$O(1)$ Delta SQLite WAL** (`b.index >
| **Empty-Wallet Search DoS** | Querying random empty addresses to force full-chain scans | Indexed SQLite `tx_lookup` query withou
| **Mempool Spam / Botnet Flood** | Single wallet or botnet flooding 10,000 pending txs | `MAX_PENDING_TX_PER_WALLET = 25`, `MAX_GL
| **Datacenter Sybil Node Attack** | Spawning 50 validator nodes in same cloud subnet to drain 30% quorum pool | `_extract_node_hos
| **Welcome Faucet Sybil Drain** | Automated wallet creation to farm 1.00 BCT bonus | Per-IP rate limits (3/min, 10/hr) + Global or
| **Frontend Tampering / CDN Hijack** | Modified `index.html` or `sw.js` served to steal keys | On-chain `frontend_release` SHA-256
```

---

14. Roadmap & Protocol Evolution

- **v1.0 – v1.6 (Completed):** Mainnet Genesis, 70/30 PoW + Validator Quorum consensus, LockTrace Space SocialFi royalties, `.bct` BNS registry & marketplace, Base Mainnet `wBCT` 1:1 cross-chain bridge, Uniswap v4 liquidity pool.

- **v1.9 (Completed):** On-chain SHA-256 + IPFS CIDv1 Frontend Release Integrity Shield, Quote-Repost attribution cards, Explorer BNS/wallet full-history indexing, Persistent Mainnet Security Discrepancy Alert System.

- **v2.0.0 (Current Production Release):** 3-Phase Atomic CAS Non-Blocking Block Producer, $O(1)$ Incremental Delta SQLite WAL Engine, $O(\Delta)$ Multi-Worker Sync, Anti-DDoS Per-Wallet Mempool Quotas, IPv4 `/24` Sybil Subnet Hardening, and Stat-Memoized Integrity Gate.

- **v2.5+ (Future Research):** Optional zk-SNARK shielded transfers, decentralized encrypted direct messaging (E2EE ECDH over SECP256k1), and native cross-chain HTLC atomic swaps.

References

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.*

2. Koblitz, N. (1987). *Elliptic Curve Cryptosystems.* Mathematics of Computation, 48(177), 203–209.

3. Pornin, T. (2013). *RFC 6979: Deterministic Usage of the Digital Signature Algorithm (DSA) and ECDSA.* IETF.

4. Palatinus, M., et al. (2013). *BIP-0039: Mnemonic Code for Generating Deterministic Keys.*

5. Castro, M., & Liskov, B. (2002). *Practical Byzantine Fault Tolerance and Proactive Recovery.* ACM TOCS.

6. Merkle, R. C. (1987). *A Digital Signature Based on a Conventional Encryption Function.* CRYPTO '87.